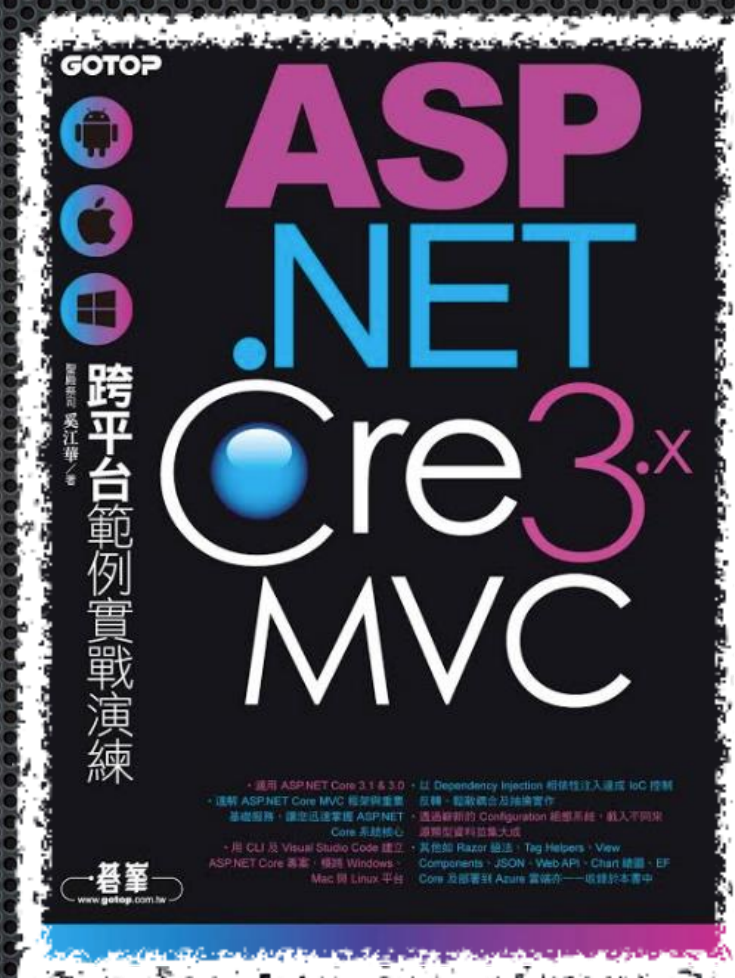


第07章

用Razor、Partial View 及 C# 語法增強View戰鬥力



作者：聖殿祭司。奚江華

大綱(1)

7-1 Razor語法概觀

7-2 十五條Razor語法規則

7-3 Razor判斷式與流程控制

7-4 以Razor語法判斷成績高低並標示不同顏色之實例

7-5 以Local function與@functions在View 中宣告方法

7-6 在View定義Razor樣板

大綱(2)

7-7 View 以@inherits 繼承自訂RazorPage類別

7-8 建立可重複使用的Partial View部分檢視

7-9 呼叫Partial View非同步與同步語法

7-10 Controller與Partial View結合 EF Core資料庫存取

Razor 語法概觀

7-1

Razor = HTML + C#混合體

主體

用@符號切換C#

Razor Markup語法範例

@{

var Name = "Kevin";

var Height = 180;

var Weight = 75;

}

C#程式區塊

HTML的<p>

<p>我的名字: @Name </p>

<p>我的身高: @Height </p>

<p>我的體重: @Weight </p>

Razor Inline表達式

C#變數

什麼是Razor？

- Razor Markup Syntax
- Razor = HTML + C#混合體的Markup標記語法
- HTML是主體，C#必須用@符號作切換
- 將C#程式嵌入到View中的HTML之Markup語法
- 將Server Side的程式嵌入到Web Pages中

Razor語法轉換成Class類別

```
@{  
    var version = "這是ASP.NET Core MVC";  
}  
  
<div>.NET Core Mvc 版本 : @version </div>
```

Razor語法

```
internal sealed class Views_Labs_RazorToCSharp :  
    global::Microsoft.AspNetCore.Mvc.Razor.RazorPage<dynamic>  
{  
    0 references  
    public async override global::System.Threading.Tasks.Task ExecuteAsync()  
    {  
        var title = "這是ASP.NET Core MVC";  
        WriteLiteral("\r\n<div>.NET App : ");  
        Write(title);  
  
        WriteLiteral("</div>\r\n\r\n");  
    }  
}
```

Class類別


```
namespace AspNetCoreGeneratedDocument
{
    using System;
    using System.Collections.Generic;
    using System.Linq;
    using System.Threading.Tasks;
    using Microsoft.AspNetCore.Mvc;
    using Microsoft.AspNetCore.Mvc.Rendering;
    using Microsoft.AspNetCore.Mvc.ViewFeatures;
    using Mvc7_Razor.Models;
    0 references
    internal sealed class Views_Labs_RazorToCSharp :
        | | | | global::Microsoft.AspNetCore.Mvc.Razor.RazorPage<dynamic>
    {
        0 references
        public async override global::System.Threading.Tasks.Task ExecuteAsync()
        {
            var title = "這是ASP.NET Core MVC";
            WriteLiteral("\r\n<div>.NET App : ");
            Write(title);

            WriteLiteral("</div>\r\n\r\n");
        }
    }
}
```

Mvc7_Razor/obj/Debug/net7/.../RazorToCSharp.cshtml.g.cs

Razor File

- Razor File為**.cshtml**，即View檢視檔
- Razor File又稱Razor View、View Template
- Razor中預設語言是HTML
- 如欲嵌入使用C#，需使用@符號
- 最終全部Render成為HTML輸出

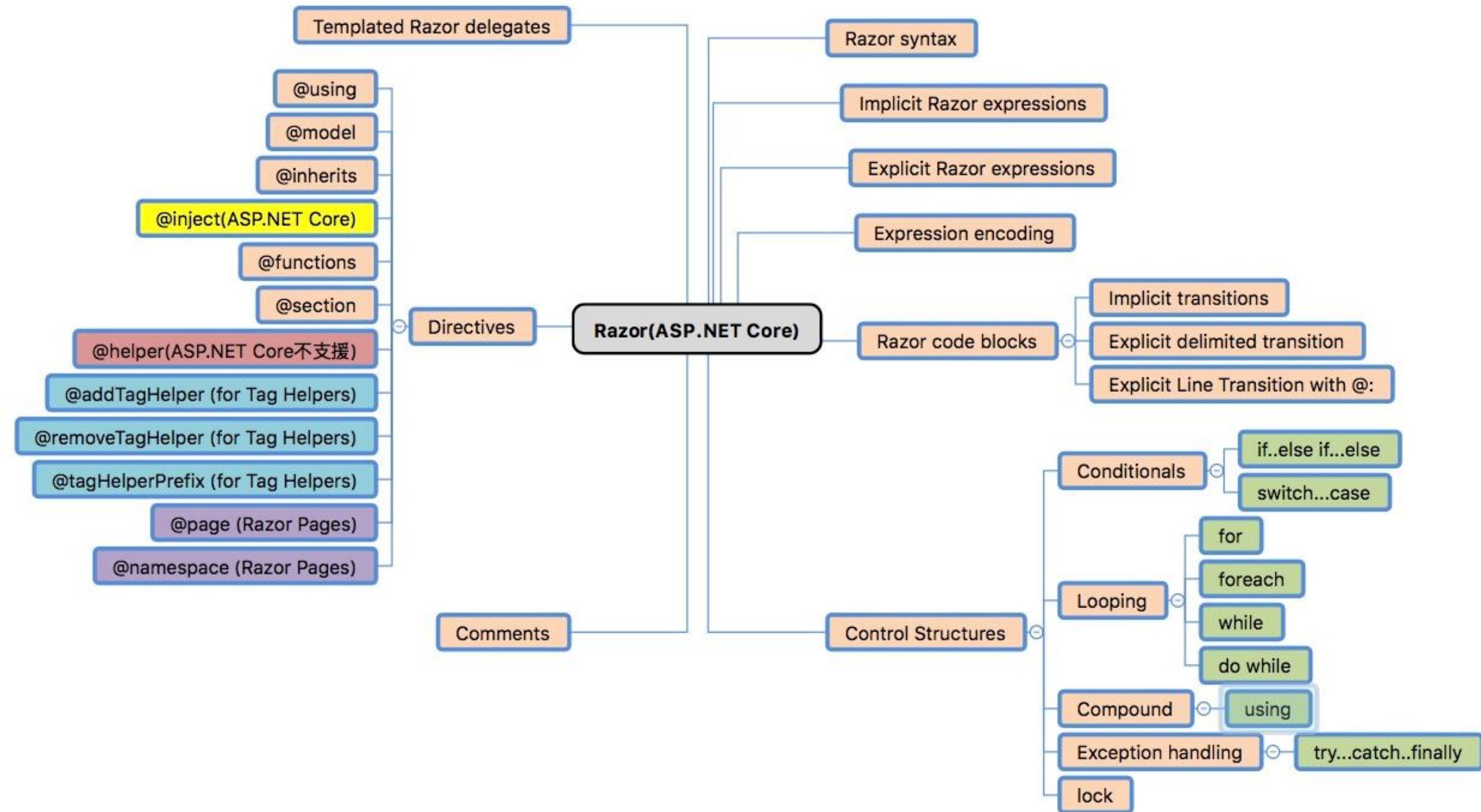
Razor View 在 ASP.NET Core 和 ASP.NET MVC 5 有何不同？

- Razor 語法支援新的關鍵字，如@page、namespace、inject 等功能
- 在 View 中可使用@function 和 local function
- ASP.NET Core 不支援 ASP.NET MVC 5 的 App_Code，所以無法將functions 建立成全域
- 不再支援 helper 關鍵字，因此 ASP.NET MVC 5 以 helper 設計的功

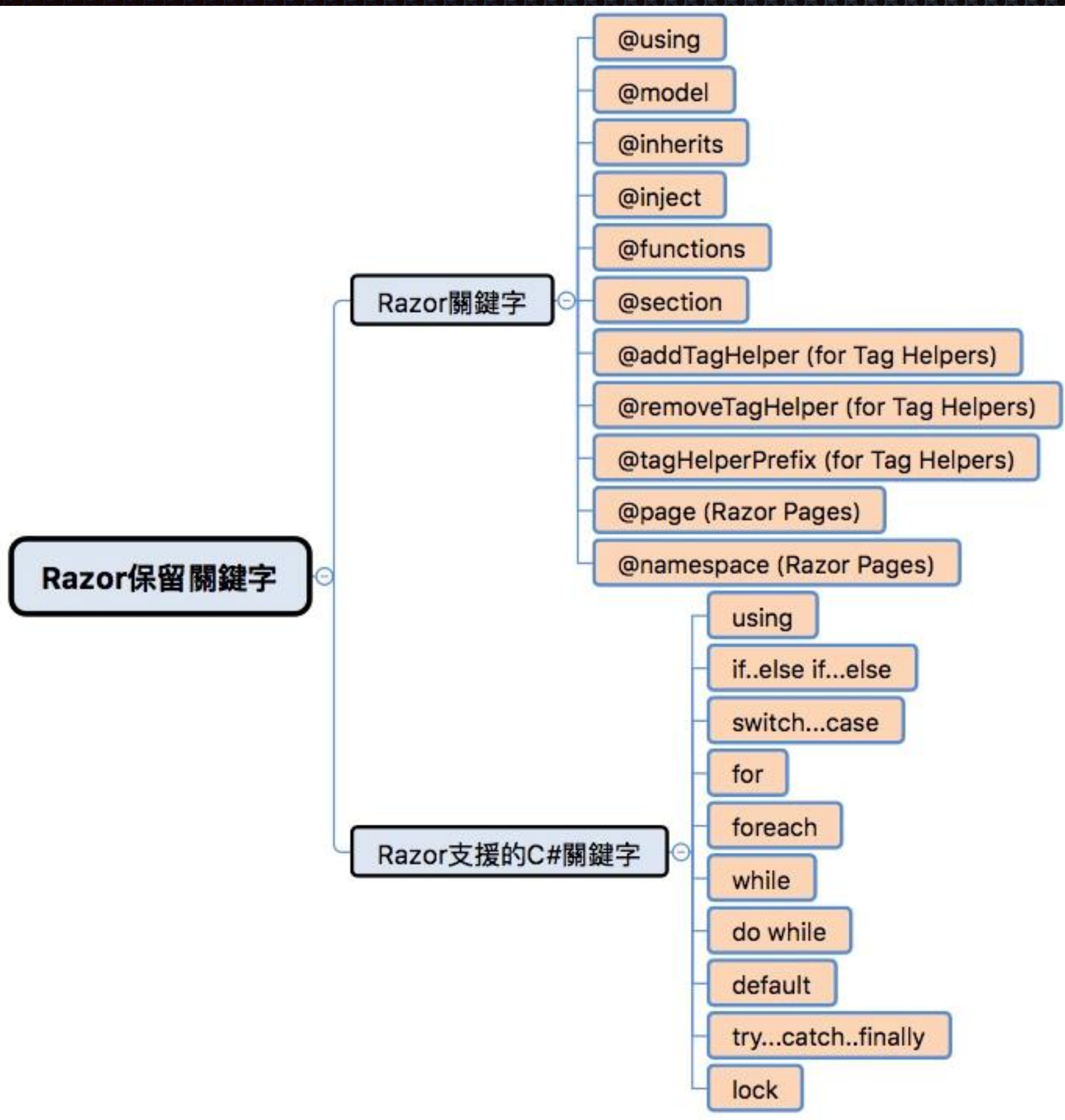
Razor View 在 ASP.NET Core 和 ASP.NET MVC 5 有何不同？

- 某些功能無法直接沿用到 ASP.NET Core，必須改成其他機制，例如 PartialView 或 View Components
- 支援全新的 Tag Helpers 和 View Components
- Scaffolding 產生的 Razor View 檢視樣板，會優先使用 Tag Helpers，其次才是 HTML Helpers，二者存在著替代情，但又相輔協作
- View 可透過 inject 關鍵字支援相依性注入
- 在 View 頁面透過 inject 注入直接調用 Configuration、Option

Razor 語法支援功能



Razor 支援的保留關鍵字



Razor兩類關鍵字

- Razor 關鍵字

page、namespace、functions、inherits、model、section 六個關鍵字是 Razor 創造的，用來支持 Razor 語法所需功能

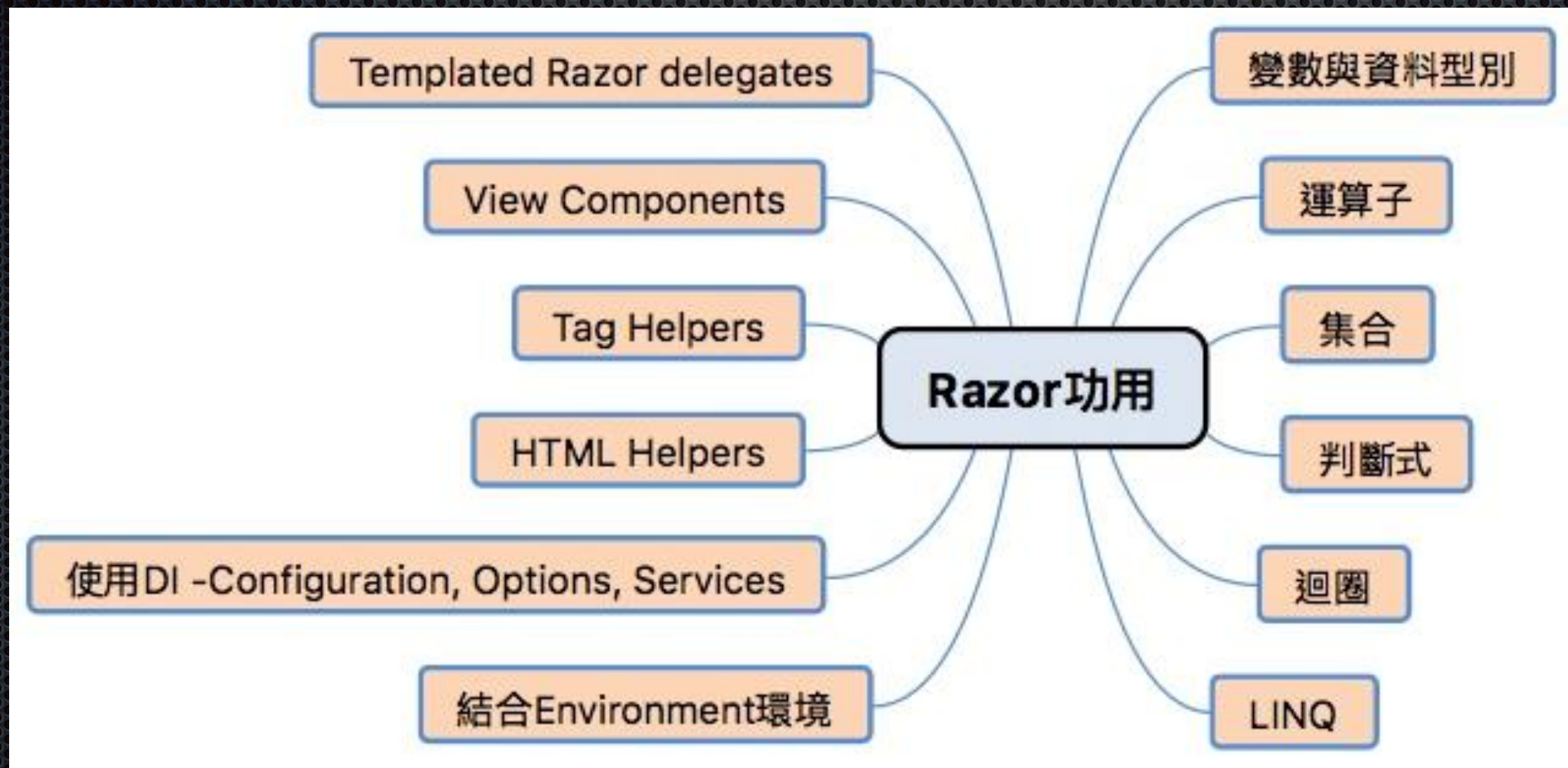
- C#關鍵字

這是源自既有的 C#，不是 Razor 所創造。Razor 支援常用的 C#關鍵字，但有些像class關鍵字就不支援

十五條Razor語法規則

7-2

Razor語法功用



規則 1：以@符號作為 C#的開頭

- Razor Markup = HTML + C#
- 預設為HTML
- 遇到單一@符號開頭，就解析為 C#語法

規則 2：以@{...}宣告單行 C#程式

```
@{ var City = "Taipei"; }
```

```
@{ var PostalCode = 110; }
```

@*以下用明確型別宣告變數也可以*@

```
@{ string city = "Taoyuan"; }
```

```
@{ int postalCode = 334; }
```


規則 3：以@{...}宣告多行 C#程式

@{

```
var Name = "Kevin";
```

```
var Height = 180;
```

```
var Weight = 75;
```

}

規則 4：C#的 Inline 表達式

<p>我的名字: @Name </p>

<p>我的身高: @Height </p>

<p>我的體重: @Weight </p>

<p>居住城市: @City </p>

<p>郵遞區號: @PostalCode </p>

HTML 輸出結果為：

我的名字: Kevin

我的身高: 180

我的體重: 75

居住城市: Taipei

郵遞區號: 110

以@符號顯示變數值的輸出

- 以@符號顯示變數值，ASP.NET 一律將變數值做 HTML 編碼，輸出成純文字
- 例如@("<h1>MVC</h1>")會輸出
<h1>MVC</h1>
- HTML編碼最大用意在於安全性，可以防止網頁攻擊程式嵌入到HTML，編碼成純文字可使其不作用

規則 5：C#程式區塊中的 HTML 隱式轉換

```
@{  
    var LeapYear = DateTime.IsLeapYear(DateTime.Now.Year);  
    <p>今年是否為閏年： @LeapYear </p>  
}
```

HTML

C#

規則 6：C#關鍵字區分大小寫

```
@{  
    var MyName = "聖殿祭司";  
    var myName = "奚江華";  
}
```

<p>

筆名: @MyName

姓名: @myName

</p>

規則 7：單行註解 - @*...*@

@*這是單行註解*@

規則 8 : 多行註解 - @*...*@

@*多行註解

也有支援*@

規則 9：Razor 隱性表達式 - @符號

<p>現在的時間是： @DateTime.Now </p>

規則 10：Razor 明確表達式 - @(...)符號

<p>兩週前我出國去玩，出發日期是：

@((DateTime.Now - TimeSpan.FromDays(14)).ToShortDateString()) </p>

<p>3+7 的結果是: @(3 + 7) </p>

規則 11：以文字顯示@符號，需用@@表示

<p>

@@_@@

<p>

<p>

但 Email 和超連結例外

我的電子郵件: dotnetcool@gmail.com

Service@domain.com

</p>

HTML:

@_@

但 Email 和超連結例外

我的電子郵件: dotnetcool@gmail.com

Service@domain.com

規則 12：字串變數中的雙引號顯示

```
@{ var word = @"子曰："""三人行，必有我師焉"""...";}
```

```
<p>@word</p>
```

HTML:

子曰： "三人行，必有我師焉"...

規則 13：用@(...)將 HTML 或 JS 編碼成純文字

```
@{ var msg = @"<button type='button' onclick='alert(\"\"Hi JavaScript\"\")'> Raw  
原始字串,不做 HTML 編碼</button>"; }
```

```
<p>@(msg)</p>
```

```
@("<span>Hello MVC!</span>") <br />
```

HTML純文字：

```
<button type='button' onclick='alert("Hi JavaScript")'> Raw 原始字串, 不做 HTML 編  
碼</button>
```

```
<span>Hello MVC!</span>
```

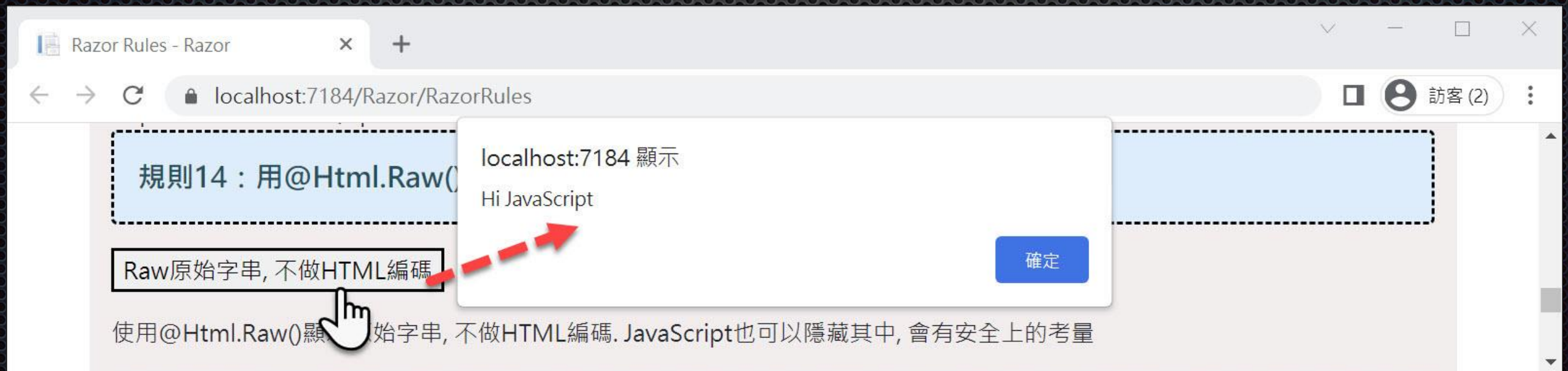
因 HTML 實際程式中<>符號被編碼成<和>, 使其變成純文字：

```
<p>&lt;button type=&#x27;button&#x27; onclick=&#x27;alert(&quot;Hi  
JavaScript&quot;)&#x27;&gt; Raw&#x539F;&#x59CB;&#x5B57;&#x4E32;,  
&#x4E0D;&#x505A;HTML&#x7DE8;&#x78BC;&lt;/button&gt;</p>  
&lt;span&gt;Hello MVC!&lt;/span&gt; <br />
```


規則 14：用@Html.Raw()顯示原始字串

@{ var msg = @"<button type='button' onclick='alert(\"\"Hi JavaScript\"\")'> Raw 原始字串,
不做 HTML 編碼</button>"; }

<p>@Html.Raw(msg)</p>



規則 15：磁碟路徑表示法

```
@{ var filePath = @"c:\Mvc7_Razor\Mvc7_Razor"; }
```

```
<p>磁碟路徑: @filePath</p>
```

HTML 輸出：

磁碟路徑: c:\Mvc7_Razor\Mvc7_Razor



Razor語法規則練習

<Views/Razor/RazorRules.cshtml>

Razor 判斷式與流程控制

7-3

if...else 條件判斷式

7-3-1

Views/Razor/RazorStatement.cshtml

if...else條件判斷式

```
@{  
    
}
```

```
var score = 95;
```

宣告score變數

```
@if (score < 60)
```

```
{
```

```
<p>@(score)分：成績不及格!</p>
```

```
}
```

```
else if (score >= 60 && score < 85)
```

```
{
```

```
<p>@(score)分：成績及格，分數尚可.</p>
```

```
}
```

```
else if (score >= 85 && score < 95)
```

```
{
```

```
<p>@(score)分：成績及格，分數中上.</p>
```

```
}
```

```
else
```

```
{
```

```
<p>@(score)分：成績及格，分數優異!</p>
```

```
}
```

if判斷並輸出結果

switch...case 判斷式

7-3-2

switch...case判斷式

```
@{ var portNumber = 587; }
```

宣告portNumber變數

```
@switch (portNumber)  
{
```

switch...case判斷並輸出結果

```
    case 80:
```

```
        <p>@portNumber，這是HTTP使用的Port號碼</p>
```

```
        break;
```

```
    case 110:
```

```
        <p>@portNumber，這是POP3使用的Port號碼</p>
```

```
        break;
```

```
    case 143:
```

```
        <p>@portNumber，這是IMAP使用的Port號碼</p>
```

```
        break;
```

```
    case 443:
```

```
        <p>@portNumber，這是HTTPS使用的Port號碼</p>
```

```
        break;
```

```
    case 587:
```

```
        <p>@portNumber，這是SMTP使用的Port號碼</p>
```

```
        break;
```

```
    default:
```

```
        <p>@portNumber，這裡沒有記載這個Port號碼的用途</p>
```

```
        break;
```

```
}
```


Local function + switch...case

```
@{  
    string ScoreComment(int score)  
    {  
        string comment = "";  
  
        switch (score)  
        {  
            case int s when s <= 60:  
                comment = $"{s}分：成績不及格!";  
                break;  
            case int s when s >= 60 && s < 85:  
                comment = $"{score}分：成績及格，分數尚可.";  
                break;  
            case int s when s >= 85 && s < 95:  
                comment = $"{score}分：成績及格，分數中上.";  
                break;  
            default:  
                comment = $"{score}分：成績及格，分數優異!";  
                break;  
        }  
  
        return comment;  
    };  
}
```

在View中定義的Local function

依分數判斷評語

回傳評語

<h1>@ScoreComment(score)</h1>

呼叫Local function

Local function + switch expression

Local function

Switch expression

```
@{  
    string PortFunction(int port) =>  
    {  
        port switch  
        {  
            80 => $"{port}, 這是HTTP使用的Port號碼",  
            110 => $"{port}, 這是POP3使用的Port號碼",  
            143 => $"{port}, 這是IMAP使用的Port號碼",  
            443 => $"{port}, 這是HTTPS使用的Port號碼",  
            587 => $"{port}, 這是SMTP使用的Port號碼",  
            _ => $"{port}, 這裡沒有記載這個Port號碼的用途"  
        };  
    }  
}
```

```
<p>@PortFunction(80)</p>  
<p>@PortFunction(443)</p>  
<p>@PortFunction(587)</p>
```

呼叫Local function

Local function獨立成Partial View重複使用

```
@{
    var portNumber = (int)ViewData["PortNumber"];

    string Port(int port) =>
        port switch
        {
            80 => $"{port}, 這是HTTP使用的Port號碼",
            110 => $"{port}, 這是POP3使用的Port號碼",
            143 => $"{port}, 這是IMAP使用的Port號碼",
            443 => $"{port}, 這是HTTPS使用的Port號碼",
            587 => $"{port}, 這是SMTP使用的Port號碼",
            _ => $"{port}, 這裡沒有記載這個Port號碼的用途"
        };
}
```

<p>@Port(portNumber)</p>

Views/Shared /_PortFunctionPartial.cshtml

呼叫Partial View

傳遞資料給Partial View

```
@await Html.PartialAsync("_PortFunctionPartial", ViewData["PortNumber"] = 80)
@await Html.PartialAsync("_PortFunctionPartial", ViewData["PortNumber"] = 110)
@await Html.PartialAsync("_PortFunctionPartial", ViewData["PortNumber"] = 143)
```


for 迴圈

7-3-3

定義匿名型別陣列資料

```
<!--宣告匿名型別陣列-->
```

```
@{  
    var ports = new[]  
    {  
        new { portNum = 80 , description = "這是HTTP使用的Port號碼"},  
        new { portNum = 110, description = "這是POP3使用的Port號碼"},  
        new { portNum = 143, description = "這是IMAP使用的Port號碼"},  
        new { portNum = 443, description = "這是HTTPS使用的Port號碼"},  
        new { portNum = 587, description = "這是SMTP使用的Port號碼"}  
    };  
}
```


用for迴圈讀取陣列資料

```
<table class="table table-bordered table-striped">
  <thead class="thead-dark">
    <tr>
      <th>Port號</th>
      <th>用途說明</th>
    </tr>
  </thead>
  <tbody>
    @for (int i = 0; i < ports.Length; i++)
    {
      <tr>
        <td>@ports[i].portNum</td>
        <td>@ports[i].description</td>
      </tr>
    }
  </tbody>
</table>
```


foreach陳述式

7-3-4

用foreach迭代陣列資料

```
<ul>  
    @foreach (var p in ports)  
    {  
        <li>@p.portNum · @p.description</li>  
    }  
</ul>
```


while 陳述式

7-3-5

用while陳述式讀取陣列資料

```
@{ var index = 0; }  
  
@while (index < ports.Length)  
{  
  
    var port = ports[index];  
  
    <p> @port.portNum, @port.description </p>  
  
    @*@: @port.portNum, @port.description <br />*@  
  
    index++;  
  
}
```


do...while 陳述式

7-3-6

用do...while讀取陣列資料

```
@{ var idx = 0;}

@do

{

    var port = ports[idx];

    @: @port.portNum, @port.description <br />

    idx++;

} while (idx < ports.Length);
```


try...catch...finally 陳述式

7-3-7

try...catch...finally陳述式

```
@try
{
    throw new InvalidOperationException("擲出
        InvalidOperationException例外");
}
catch (InvalidOperationException ex)
{
    <p>捕捉到InvalidOperationException例外錯誤訊息: @ex.Message</p>
}
catch (Exception ex)
{
    <p>捕捉到Exception例外錯誤訊息: @ex.Message</p>
}
finally
{
    <p>這是最終的finally陳述式</p>
}
```




Razor陳述式

<Views/Razor/RazorStatement.cshtml>

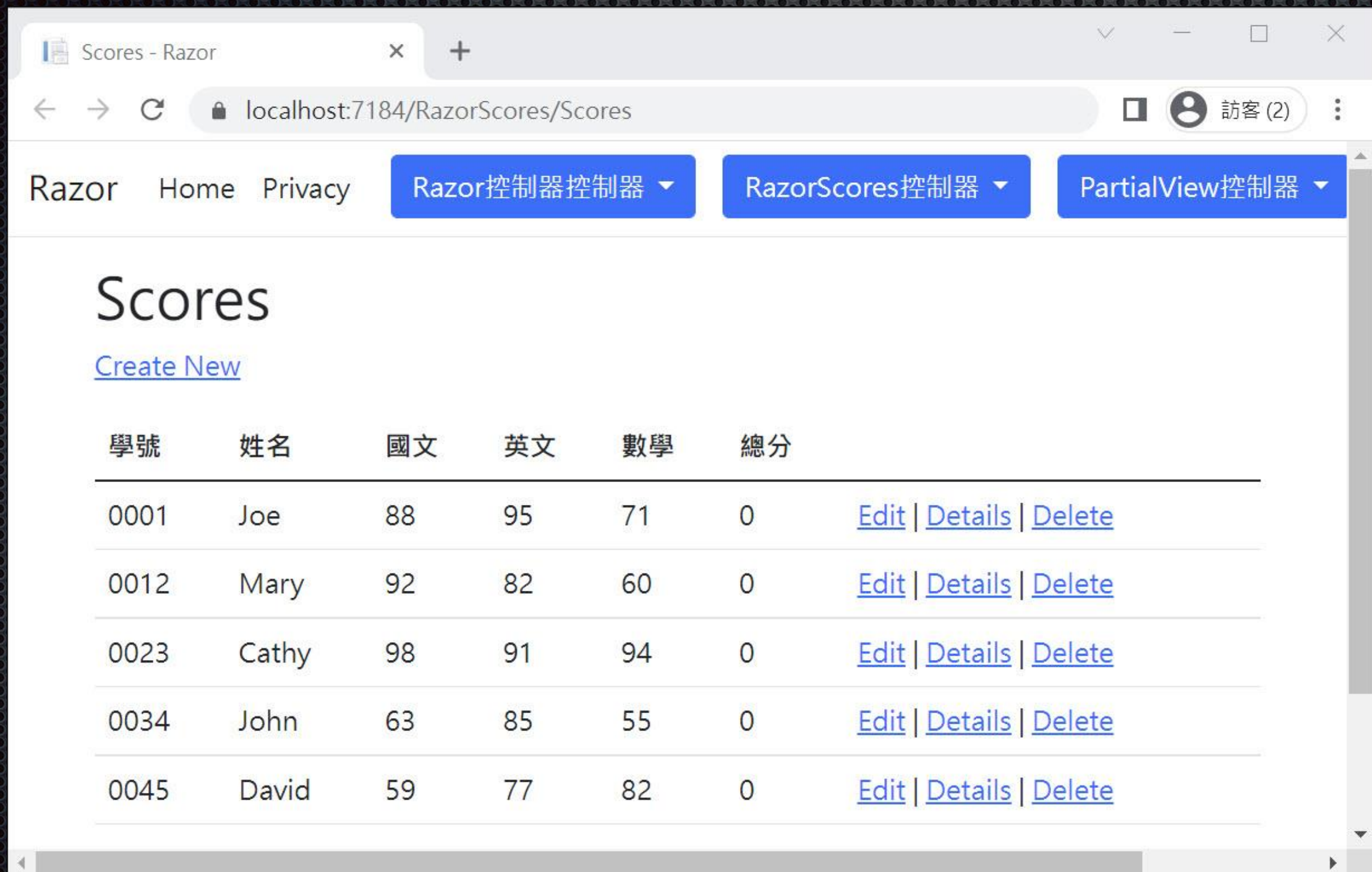
以Razor語法判斷成績高低
並標示不同顏色之實例

7-4

以兩範例逐步體驗Razor

- 第一個範例7-1：先製作學生成績列表的原型程式，也就是先不用 Razor強化View
- 第二個範例7-2：以Razor語法判斷學生成績高低及找出總分最高者

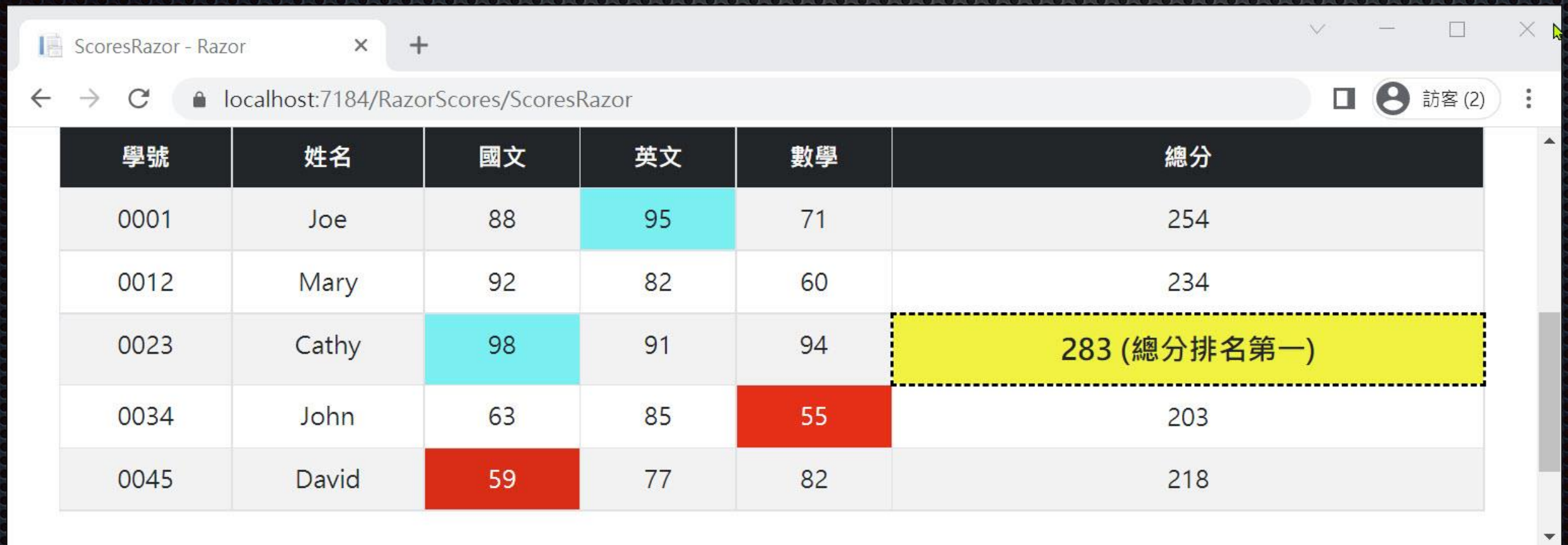
建立學生成績列表的原型



The screenshot shows a web browser window with the title 'Scores - Razor'. The address bar displays 'localhost:7184/RazorScores/Scores'. The page features a navigation bar with links for 'Razor', 'Home', and 'Privacy', followed by three blue buttons: 'Razor控制器控制器', 'RazorScores控制器', and 'PartialView控制器'. Below the navigation bar, the main heading 'Scores' is displayed, followed by a 'Create New' link. A table lists student records with columns for '學號' (Student ID), '姓名' (Name), '國文' (Chinese), '英文' (English), '數學' (Math), and '總分' (Total Score). Each row includes links for 'Edit', 'Details', and 'Delete'.

學號	姓名	國文	英文	數學	總分	
0001	Joe	88	95	71	0	Edit Details Delete
0012	Mary	92	82	60	0	Edit Details Delete
0023	Cathy	98	91	94	0	Edit Details Delete
0034	John	63	85	55	0	Edit Details Delete
0045	David	59	77	82	0	Edit Details Delete

以Razor語法判斷成績高低並找出總分最高者



學號	姓名	國文	英文	數學	總分
0001	Joe	88	95	71	254
0012	Mary	92	82	60	234
0023	Cathy	98	91	94	283 (總分排名第一)
0034	John	63	85	55	203
0045	David	59	77	82	218



範例7-1 製作學生考試程績列表

範例7-2 以Razor語法判斷學生成績
高低及找出總分最高者

範例7-3 在 View 中用 Razor、C# 8
及 LINQ 找出總分最高者

以Local function 與@functions 在View中宣告方法

7-5

@functions

- @function 是在 View 中宣告 Fields、Properties 和 Methods 成員，然後 View 就能調用這些成員
- 且編譯後，它們會轉換成 C#類別成員


```
@inject Microsoft.Extensions.Configuration.IConfiguration config
```

以inject注入組態

```
@functions  
{
```

```
    public string Name = "聖殿祭司";
```

Field

```
    public string RealName { get; } = "奚江華";
```

Property

```
    public string GetPhoneNumber()  
{  
        return "0925-123-123";  
    }
```

Method

```
    public string BookTitle()  
{  
        return @config["Book:Title"] ?? "查無資料";  
    }
```

```
<h2>書名 : @BookTitle()</h2>
```

呼叫port function

```
<p>筆名 : @Name</p>
```

```
<p>姓名 : @RealName</p>
```

```
<p>電話 : @GetPhoneNumber()</p>
```

Views/Razor/RazorFunctions.cshtml


```

@functions{
    private void PersonInfo()
    {
        <div>
            <h1>@Name</h1>
            <p>姓名:@RealName , 電話: @GetPhoneNumber()</p>
        </div>
    }
}

private async Task RenderCard(string name, string brief, string photo)
{
    <div class="col-xl-3 col-lg-4 col-md-6 col-sm-12">
        <div class="card">
            <div class="headshot">
                
            </div>
            <div class="card-body">
                <h5 class="card-title">@name</h5>
                <p class="card-text">@brief</p>
            </div>
        </div>
    </div>
}

}

@{
    PersonInfo();

    await RenderCard("星際大戰", "維達元帥", "Vader.jpg");
}

```

@function樣板方法含有HTML

呼叫@function樣板方法

在View定義Razor樣板

7-6

Razor 樣板(Template)

```
@{
    List<Card> Cards = new List<Card>
    {
        new Card { Name = "Elon Musk", Brief="特斯拉創辦人 伊隆·馬斯克", Photo="ElonMusk.jpg" },
        ...
    };
}

Func<dynamic, object> cardTempate = @<div class="col-xl-3 col-lg-4 col-md-6 col-sm-12">
    <div class="card">
        <div class="headshot">
            
        </div>
        <div class="card-body">
            <h5 class="card-title">@item.Name</h5>
            <p class="card-text">@item.Brief</p>
        </div>
    </div>
</div>;

<div class="row">
    @foreach (var card in Cards)
    {
        @cardTempate(card)
    }
</div>
```

Razor 樣板

呼叫 Razor 樣板

Views/Razor/RazorTemplate.cshtml

View以@inherits繼承
自訂RazorPage類別

7-7

自訂 CustomRazorPage<dynamic> 抽象類別

```
using Microsoft.AspNetCore.Mvc.Razor;

namespace Mvc7_Razor.RazorPages
{
    public abstract class CustomRazorPage<TModel> : RazorPage<TModel>
    {
        public string AppVersion { get; } = "ASP.NET Core 7";

        public string GetBookName()
        {
            return "ASP.NET Core 7 MVC範例教學實戰";
        }
    }
}
```

自訂RazorPage

RazorPages/CustomRazorPage.cs

在ViewView 以@inherits 繼承 自訂RazorPage類別

- 調用繼承而來的 GetBookName()方法與 AppVersion 屬性

```
@inherits Mvc7_Razor.RazorPages.CustomRazorPage<TModel>
```

繼承自訂的CustomRazorPage

```
@{  
    ViewData["Title"] = "InheritsRazorPage";  
}
```

<p>書籍名稱： @GetBookName()</p> 調用RazorPage方法與屬性

<p>ASP.NET Core版本： @AppVersion</p>

<p>作者： @Model.Name</p>

建立可重複使用的 Partial View部分檢視

7-8

什麼是Partial View部分檢視？

- 一般 View是設計一個大型完整的View頁面為目的
- Partial View是一小區塊，以重複使用或簡化設計為目的

```
<div class="col-xl-3 col-lg-4 col-md-6 col-sm-12">  
  <div class="card">  
    <div class="headshot">  
        
    </div>  
    <div class="card-body">  
      <h5 class="card-title">Mark Zuckerberg</h5>  
      <p class="card-text">Facebook創辦人 馬克·祖伯克</p>  
      <a href="https://goo.gl/BktGGA" class="btn btn-primary">Wiki</a>  
    </div>  
  </div>  
</div>
```

Views/Shared/_SimpleCardPartial.cshtml

Partial View接受model資料

```
@model IEnumerable<Card>
```

宣告傳入model型別

```
@foreach (var item in Model)
```

```
{
```

```
    <div class="col-xl-3 col-lg-4 col-md-6 col-sm-12">
```

```
        <div class="card">
```

```
            <div class="headshot">
```

```
                
```

```
            </div>
```

```
            <div class="card-body">
```

```
                <h5 class="card-title">@item.Name</h5>
```

```
                <p class="card-text">@item.Brief</p>
```

```
                <a href="@item.WikiUrl" class="btn btn-primary">Wiki</a>
```

```
            </div>
```

```
        </div>
```

```
    </div>
```

```
}
```

Views/Shared/_CardListPartial.cshtml

Partial View運作方式與特性

7-8-1

Partial View運作方式

Parent View
父檢視

Index.cshtml

```
@{  
    ViewBag.Title="View呼叫部分檢視"  
}
```

```
<div class="row">
```

```
@Html.Partial("RankingPartial")
```

```
@Html.Partial("NewsPartial")
```

```
</div>
```

呼叫排行榜的部分檢視

呼叫新聞的部分檢視

Partial View
部分檢視

xxx.cshtml

RankingPartial.cshtml
部分檢視

NewsPartial.cshtml
部分檢視

Partial View特性

- Partial View是一小區塊內容，供所有Views呼叫使用
- Partial View 延伸檔名與 View 相同，都是.csthml
- 呼叫Partial View的View，又稱為Parent View父檢視
- Partial View 若被Views 呼叫，Partial View 的內容就會Render加入到Parent View中
- Partial View 可放在個別的 View 資料夾，同一個資料夾的 Parent View會優先參考使用。也可放在 Views/Shared 資料夾，讓所有資料夾的Parent View 共用
- 因此 Parent View 尋找 Partial View 的順序，是先本身所在的資料夾搜尋，若找不到的話，才會到 Shared 資料夾中尋找

Partial View和View的區別

- Partial View 不會執行_ViewStart.cshtml
- 若在 Partial View 中宣告 Section，Section 不會出現在 Parent View，等於是無法作用
- 雖然 Partial View 可加入<style />和<script />，但是若被呼叫多次，這些css和js也會重複產生多次
- Partial View 支援鏈狀(chained)呼叫，就是可以再層層呼叫其他的 Partial View

Partial View的資料傳遞方式

7-8-2

Partial View使用注意事項

- Parent View 父檢視呼叫 Partial View 可分成非同步與同步的語法，請優先使用非同步語法，盡量避免使用同步語法
- 在呼叫的同時，還可用 ViewData 或 model 物件傳遞資料給 Partial View
- Partial View 的命名，會建議用_底線開頭，並以 Partial 結尾，例如_CardPartial.cshtml

以ViewData傳遞資料給Partial View

```
<partial name="_MediaPartial" view-data="ViewData" />

@await Html.PartialAsync("_MediaPartial", ViewData)

@{ await Html.RenderPartialAsync("_MediaPartial", ViewData); }
```

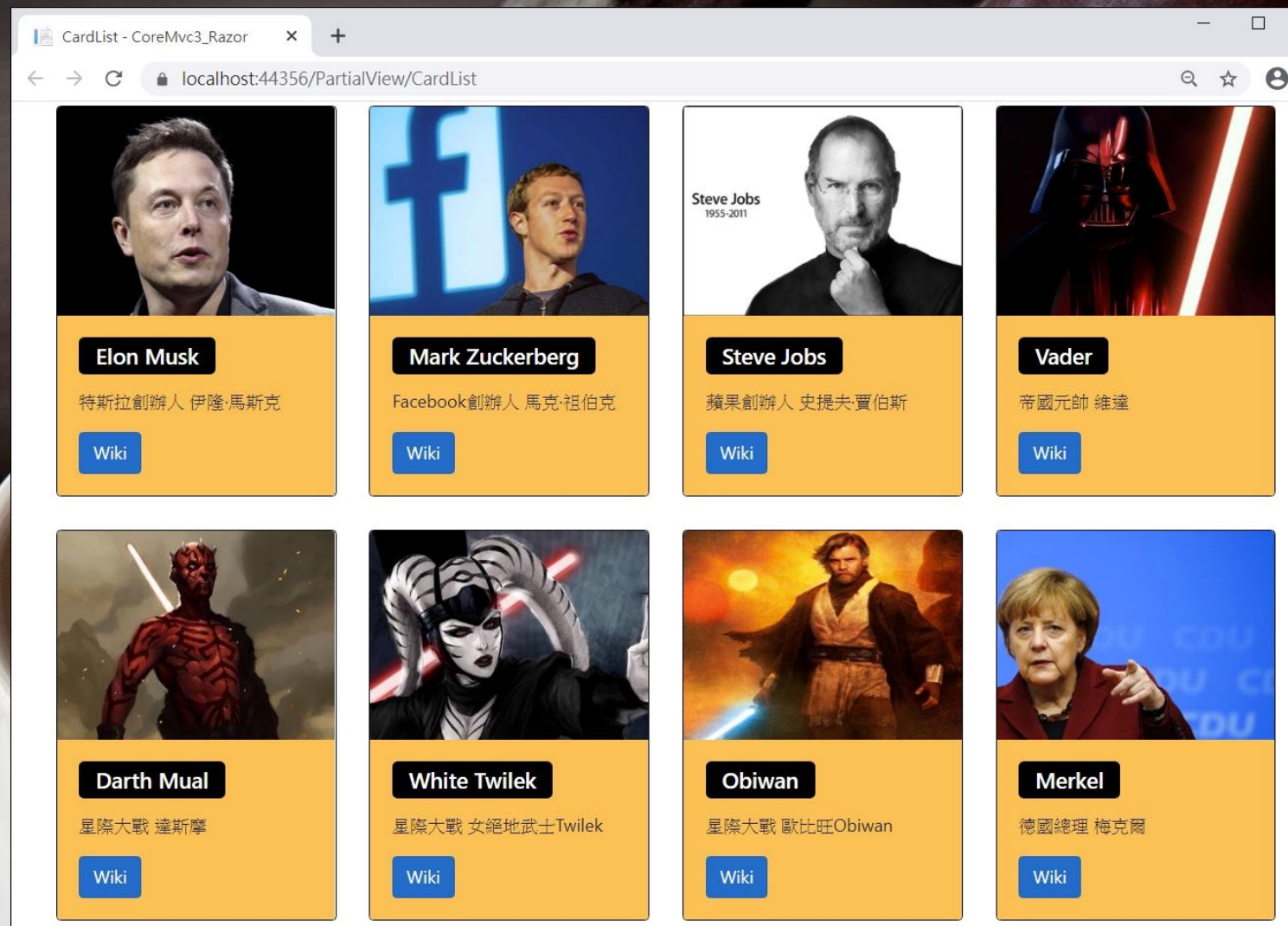
Views/PartialView/PassViewData2PartialView.cshtml

- Partial View初始時，會收到 Parent View傳來的 ViewDataDictionary 複本
- 但 Partial View 更動 ViewData 資料，ViewData 不會更新回 ParentView
- Partial View 回傳時，它的 ViewData 便會消失

以 model 物件傳遞資料給 Partial View

```
<partial name="_CardListPartial" model="@Model" />  
  
<partial name="_CardListPartial" for="@Model" />  
  
@await Html.PartialAsync("_CardListPartial", Model)  
  
@{ await Html.RenderPartialAsync("_CardListPartial", Model); }
```

Views/PartialView/PassModel2PartialView.cshtml



範例 7-4

將人物牌卡製作成 **Partial View**，供所有 **View** 呼叫使用

範例 7-5

傳遞 **model** 資料到 **Partial View**，動態生成不同的牌卡

呼叫Partial View 非同步與同步語法

7-9

呼叫 Partial View 非同步與同步完整語法

7-9-1

呼叫與傳遞View Data

- 非同步語法(建議)

```
<partial name="_MediaPartial" view-data="ViewData" />  
@await Html.PartialAsync("_MediaPartial", ViewData)  
@{ await Html.RenderPartialAsync("_MediaPartial", ViewData); }
```

- 同步語法(不建議)

```
@Html.Partial("_MediaPartial", ViewData)  
@{ Html.RenderPartial("_MediaPartial", ViewData); }
```


呼叫與傳遞Model

- 非同步語法(建議)

```
<partial name="_CardListPartial" model="@Model" />
<partial name="_CardListPartial" for="@Model" />

@await Html.PartialAsync("_CardListPartial", Model)
@{ await Html.RenderPartialAsync("_CardListPartial", Model); }
```

- 同步語法(不建議)

```
@Html.Partial("_CardListPartial", Model)

@{ Html.RenderPartial("_CardListPartial", Model); }
```


RenderPartialAsync 和 RenderPartial 方法之特點

7-9-2

RenderPartialAsync和RenderPartial方法

```
@{ await Html.RenderPartialAsync("_CardListPartial", Model); }  
  
@{ Html.RenderPartial(_"CardListPartial", Model); }
```

- 它們轉譯部分檢視不會傳回 `IHtmlContent`，而是將轉譯輸出直接串流給 `Response` 回應。且因不回傳結果，所以必須在 **Razor 程式碼區塊** 內呼叫它們
- 相較於 `PartialAsync` 和 `Partial` 方法，有時可能會有較佳的效能

A close-up photograph of a potter's hands, covered in wet clay, shaping a light-colored clay pot on a pottery wheel. The wheel is in motion, creating a blurred background. The pot has a wide, flared rim and a rounded body. The potter's hands are positioned to smooth the rim and the upper body of the pot. The lighting is dramatic, highlighting the texture of the clay and the potter's skin.

範例 7-6

傳遞 ViewData 資料到 Partial View 的非同步語法

範例 7-7

傳遞 Model 資料到 Partial View 的非同步語法

Controller與Partial View 結合EF Core資料庫存取

7-10

範例 7-8 Partial View 結合 EF Core 資料庫存取

- ① 建立Card 模型
- ② 安裝EF Core相關套件
- ③ 建立 CardContext.cs 及種子資料
- ④ 在 appsettings.json 設定資料庫連線
- ⑤ 在 DI Container 註冊 CardContext
- ⑥ 執行 Migration 命令
- ⑦ 在PartialView控制器注入CardContext及建立CardListDB動作方法



範例 7-8

Partial View 結合 EF Core 資料庫存取

The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹**、**ASP.NET Core 3.x MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw